

claude-windows / 8d4866bf-8edd-4da9-909e-022ddee12675

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\8d4866bf-8edd-4da9-909e-022ddee12675\subagents\workflows\wf_e27060ad-65f\agent-a2bb777a153c6f9f1.jsonl`  
- SHA-256: `16ff73344a28299e03c698e4fa7fcb80bb535de1e7d8eafdf92e361f3b559af9`  
- Source modified: `2026-06-19T16:41:24+00:00`  
- Imported at: `2026-07-05T16:48:25+00:00`  
- project: `wf_e27060ad-65f`  
- session_id: `8d4866bf-8edd-4da9-909e-022ddee12675`
```

Transcript

1. user (2026-06-19T16:40:08.111Z)

You are adversarially reviewing a technical claim in a proposed PR. Verify it against REAL code; quote exact lines. Try BOTH to confirm and to refute.

```
PR CLAIM A ? "Subtle Calculation Artifact in _spread (rally_gate.py)":  
> The _spread function calculates a robust spread (ignoring bottom 5% / top 5%):  
>   lo = s[max(0, int(0.05 * (len(s) - 1)))]  
>   hi = s[min(len(s) - 1, int(0.95 * (len(s) - 1)))]  
> Because int() truncates rather than rounds, for very short arrays (< 20 items) the 95th  
percentile is simply the second-to-last item, and for len(s) == 2 the spread evaluates to  
strictly 0.0.  
> Impact: doesn't crash because play axis is estimated over the entire video's thousands of  
points; but if reused on small window slices in future it will quietly collapse spread to zero.  
The PLAN also escalates this to "potential divide-by-zero or zero-spread" and proposes: "Modify  
_spread to use math.ceil() or properly enforce a minimum subset size before trimming  
percentiles, ensuring it never inadvertently calculates a spread of 0 for non-identical values."
```

TASKS:

1. Read backend/pipeline/detectors/rally_gate.py around line 37 (the _spread def) and ALL its call sites (around line 63, and anywhere else). Quote the exact code.
2. Verify the len==2 ? 0.0 claim by tracing the index math (and optionally a 3-line python repro). Is the quoted code accurate to what's actually there?
3. Assess the IMPACT claim: how is _spread actually called? Is it EVER called on small slices today? Trace estimate_play_axis / the deadband usage. Is there ANY division by spread anywhere (the plan says "divide-by-zero" / "zero deadband")? If spread is 0, what actually happens downstream ? does anything break, or is it harmless?
4. Is there a SECOND _spread in backend/features/rally_seq.py:43? If so, note whether the PR is even reviewing the right one, and whether the two differ.
5. Assess the proposed fix: would math.ceil() actually fix a real problem? Is "enforce minimum subset size" well-specified? Could the fix change current production behavior (the play-axis estimate over thousands of points)?

Return the structured verdict. claim_id = "A-spread".

2. assistant (2026-06-19T16:40:10.627Z)

I'll review this claim against the real code. Let me start by reading the relevant files.

3. user (2026-06-19T16:40:11.585Z)

```
1      """Rally-start gate: suppress between-point false fires using shuttle net-crossings.  
2  
3      WASB (shuttle-only) over-detects: when a player flicks/tosses the shuttle back to  
4      the opponent for service after losing a point, that single trajectory looks like a  
5      rally. The discriminator is structural, not duration-based: a real rally has the
```

```

6 shuttle crossing the net MULTIPLE times (each shot lands on the other side); a
7 single service feed crosses -once.
8
9 This module is camera-agnostic and needs NO new model ? it works on the trajectory
10 we already produce (Frame,Visibility,X,Y). It estimates the play axis (the image
11 axis with the larger shuttle spread) and the net position (median shuttle coord on
12 that axis ? the shuttle clusters at the two ends and crosses the net region often,
13 so the median sits near the net), then counts sign changes of (coord - net) across
14 the visible points inside a candidate window, with a deadband to ignore jitter.
15
16 Gate A in the over-fire fix spectrum (B = player-stance state machine, future).
17 Pure functions only ? no I/O, unit-tested; intended as a post-filter on the
18 windows from trajectory_to_action_windows, or to fold into it later.
19 """
20 from __future__ import annotations
21
22 from dataclasses import dataclass
23 from typing import List, Optional, Sequence, Tuple
24
25 Interval = Tuple[float, float]
26
27
28 @dataclass
29 class GatedWindow:
30     start: float
31     end: float
32     crossings: int
33     visible_points: int
34     kept: bool
35
36
37 def _spread(vals: Sequence[float]) -> float:
38     """Robust spread = p95 - p5 (ignores a few outlier detections)."""
39     if len(vals) < 2:
40         return 0.0
41     s = sorted(vals)
42     lo = s[max(0, int(0.05 * (len(s) - 1)))]
43     hi = s[min(len(s) - 1, int(0.95 * (len(s) - 1)))]
44     return hi - lo
45
46
47 def _median(vals: Sequence[float]) -> float:
48     if not vals:
49         return 0.0
50     s = sorted(vals)
51     n = len(s)
52     return s[n // 2] if n % 2 else 0.5 * (s[n // 2 - 1] + s[n // 2])
53
54
55 def estimate_play_axis(points) -> Tuple[str, float, float]:
56     """Return (axis 'x'/'y', net_value, span) from visible points.
57
58     Play axis = the image axis along which the shuttle travels most (larger spread).
59     For a baseline camera that's Y (players top/bottom); for a side camera, X.
60     """
61     xs = [p.x for p in points if p.visible]
62     ys = [p.y for p in points if p.visible]
63     sx, sy = _spread(xs), _spread(ys)
64     if sx >= sy:
65         return "x", _median(xs), sx
66     return "y", _median(ys), sy
67
68
69 def count_net_crossings(window_points, axis: str, net: float, deadband: float) -> int:
70     """Count sign changes of (coord - net) across visible points, ignoring the

```

```

71     deadband zone near the net (so jitter at the net doesn't inflate the count)."""
72     crossings = 0
73     last_side = None
74     for p in window_points:
75         if not p.visible:
76             continue
77         v = (p.x if axis == "x" else p.y) - net
78         if abs(v) < deadband:
79             continue
80         side = v > 0
81         if last_side is not None and side != last_side:
82             crossings += 1
83         last_side = side
84     return crossings
85
86
87 def _points_in(points, fps: float, start: float, end: float):
88     return [p for p in points if start * fps <= p.frame <= end * fps]
89
90
91 def gate_windows(points, windows: List[Interval], fps: float,
92                 min_crossings: int = 2, deadband_frac: float = 0.06,
93                 estimate: Optional[Tuple[str, float, float]] = None) ->
List[GatedWindow]:
94     """Annotate each (start,end) window with its shuttle net-crossing count and a
95     keep decision (crossings >= min_crossings). Deadband is a fraction of the play
96     axis span. Net/axis are estimated once over the whole trajectory (stable).
97
98     `estimate` lets a caller scoring MANY windows off one trajectory pass a precomputed
99     `(axis, net, span)` (from `estimate_play_axis`) so the O(P log P) estimate isn't
redone
100     per window ? same result, avoids the per-window re-estimation."""
101     axis, net, span = estimate if estimate is not None else estimate_play_axis(points)
102     deadband = deadband_frac * span
103     out: List[GatedWindow] = []
104     for (s, e) in windows:
105         wp = _points_in(points, fps, s, e)
106         c = count_net_crossings(wp, axis, net, deadband)
107         vis = sum(1 for p in wp if p.visible)
108         out.append(GatedWindow(start=s, end=e, crossings=c, visible_points=vis,
109                               kept=c >= min_crossings))
110     return out
111
112
113 def apply_gate(points, windows: List[Interval], fps: float,
114               min_crossings: int = 2, deadband_frac: float = 0.06) -> List[Interval]:
115     """Convenience: return only the windows that pass the crossing gate."""
116     return [(g.start, g.end) for g in
117             gate_windows(points, windows, fps, min_crossings, deadband_frac) if g.kept]
118

```

4. user (2026-06-19T16:40:11.734Z)

```

1     """Per-frame rally features over a shuttle/ball trajectory ? the rally_seq feature
schema.
2
3     THE single source of truth for the Gen-0 TAL feature extraction (ADR-008): the training
4     harness (training/) and the serving path import THIS module, so train == serve by
5     construction. The fps-normalization bug fixed on 2026-06-10 (speed read ~2x smaller at
6     59.94fps than 30fps) is exactly the train/serve-skew class this placement prevents.
7
8     Pure numpy/scipy ? importable without torch/cv2. Any change to the feature semantics
9     MUST bump FEATURE_SCHEMA_VERSION; artifact manifests record the version they were
10    trained with and the serving loader refuses a mismatch.
11    """

```

```

12     from __future__ import annotations
13
14     from typing import Dict
15
16     import numpy as np
17     from scipy.ndimage import uniform_filter1d, maximum_filter1d
18
19     # Bump on ANY semantic change to build_frame_arrays/features (names, normalization,
20     # windowing, occlusion rule). Artifacts pin this; the loader hard-fails on mismatch.
21     FEATURE_SCHEMA_VERSION = 1
22
23     FEAT_NAMES = ["vis_rate", "spd_mean", "spd_max", "cross_rate", "x_std", "y_std"]
24
25     # Default windowing ? part of the schema (manifests record the values actually used).
26     DEFAULT_WIN_SEC = 0.75
27     DEFAULT_STRIDE_SEC = 0.1
28
29     FEATURE_SCHEMA = {
30         "id": "rally_seq",
31         "version": FEATURE_SCHEMA_VERSION,
32         "feat_names": FEAT_NAMES,
33         "win_sec": DEFAULT_WIN_SEC,
34         "stride_sec": DEFAULT_STRIDE_SEC,
35         # Explicit because its absence was a real bug: speed is fraction-of-frame-width
36         # PER SECOND (* fps), matching the heuristic windowing brain.
37         "fps_semantics": "per-second",
38         "occlusion_rule": "speed/crossings not computed across gaps > max(2,
39 round(0.25*fps)) frames",
40         "input": "trajectory points (frame, visible, x, y) + REQUIRED runtime metadata {fps,
41 frame_width}",
42     }
43
44     def _spread(a: np.ndarray) -> float:
45         return float(np.percentile(a, 95) - np.percentile(a, 5)) if a.size > 1 else 0.0
46
47     def build_frame_arrays(points, frame_width: float, fps: float = 30.0) -> Dict:
48         """Per-frame trajectory arrays + per-visible-frame speed and net-crossing events.
49
50         Speed is normalized to fraction-of-frame-width PER SECOND (`* fps`), matching the
51         heuristic windowing brain ? previously it was per-frame, so the same physical
52         shuttle
53         transfer
54         long
55         speed read ~2x smaller at 59.94 fps than at 30 fps, confounding cross-footage
56         on a corpus that mixes frame rates. Speed/crossings are also not computed across a
57         occlusion gap (mirrors the heuristic resetting on invisibility)."""
58         pts = sorted(points, key=lambda p: p.frame)
59         N = (pts[-1].frame + 1) if pts else 1
60         vis = np.zeros(N)
61         x = np.zeros(N)
62         y = np.zeros(N)
63         for p in pts:
64             if 0 <= p.frame < N and p.visible and p.x >= 0:
65                 vis[p.frame] = 1.0
66                 x[p.frame] = float(p.x)
67                 y[p.frame] = float(p.y)
68
69         mask = vis > 0
70         play_is_x = _spread(x[mask]) >= _spread(y[mask])
71         coord = x if play_is_x else y
72         net = float(np.median(coord[mask])) if mask.any() else 0.0
73         deadband = 0.06 * (_spread(coord[mask]) if mask.sum() > 1 else 1.0)

```

```

72     speed = np.zeros(N)
73     spd_mask = np.zeros(N)
74     cross = np.zeros(N)
75     max_gap = max(2, round(0.25 * fps)) # frames; a longer gap = occlusion break, not
motion
76     prev = None
77     for p in pts:
78         f = p.frame
79         if not (0 <= f < N and vis[f]):
80             continue
81         if prev is not None:
82             df = f - prev
83             if 0 < df <= max_gap:
84                 speed[f] = float(np.hypot(x[f] - x[prev], y[f] - y[prev]) / frame_width
/ df * fps)
85                 spd_mask[f] = 1.0
86                 c, pc = coord[f] - net, coord[prev] - net
87                 if abs(c) > deadband and abs(pc) > deadband and (c > 0) != (pc > 0):
88                     cross[f] = 1.0
89             prev = f
90     return dict(N=N, vis=vis, x=x, y=y, speed=speed, spd_mask=spd_mask, cross=cross,
fw=frame_width)
91
92
93     def features(arr: Dict, fps: float, win_sec: float = DEFAULT_WIN_SEC,
94                 stride_sec: float = DEFAULT_STRIDE_SEC):
95         """Windowed per-frame features sampled at a stride -> (X[n_samples,6],
times[n_samples])."""
96         N = arr["N"]
97         fw = arr["fw"]
98         size = max(3, int(win_sec * fps) * 2 + 1)
99         vis, speed, spd_mask, cross, x, y = (arr[k] for k in ("vis", "speed", "spd_mask",
"cross", "x", "y"))
100
101         n = uniform_filter1d(vis, size, mode="constant") # mean
visibility (== vis_rate)
102         sm = uniform_filter1d(spd_mask, size, mode="constant")
103         spd_mean = uniform_filter1d(speed, size, mode="constant") / np.maximum(sm, 1e-9)
104         spd_max = maximum_filter1d(speed, size, mode="constant")
105         cross_rate = uniform_filter1d(cross, size, mode="constant") * fps # crossings
/ second
106         xm = uniform_filter1d(x * vis, size, mode="constant") / np.maximum(n, 1e-9)
107         x2 = uniform_filter1d((x * vis) ** 2, size, mode="constant") / np.maximum(n, 1e-9)
108         x_std = np.sqrt(np.maximum(x2 - xm ** 2, 0.0)) / fw
109         ym = uniform_filter1d(y * vis, size, mode="constant") / np.maximum(n, 1e-9)
110         y2 = uniform_filter1d((y * vis) ** 2, size, mode="constant") / np.maximum(n, 1e-9)
111         y_std = np.sqrt(np.maximum(y2 - ym ** 2, 0.0)) / fw
112
113         idx = np.arange(0, N, max(1, int(stride_sec * fps)))
114         X = np.stack([n[idx], spd_mean[idx], spd_max[idx], cross_rate[idx], x_std[idx],
y_std[idx]], axis=1)
115         return np.nan_to_num(X), idx / fps
116

```

5. assistant (2026-06-19T16:40:15.100Z)

Both files are now read. Let me verify the claims by tracing the index math and checking call sites. Let me find all call sites of `\_spread` and `estimate\_play\_axis`, and confirm there's no division by spread.

6. user (2026-06-19T16:40:16.893Z)

```

backend\api\static\app.js:227:      <span><strong>Start:</strong>
${formatTime(rally.start_time)}</span>
backend\api\static\app.js:228:      <span><strong>End:</strong>

```

```

${formatTime(rally.end_time)}</span>
backend\api\static\index.html:18:                <span class="status-indicator"></span>
backend\api\static\index.html:19:                <span>Local Server Active</span>
backend\api\static\index.html:70:                <span id="rally-count-label"
class="rally-tag" style="background: rgba(255,255,255,0.06);">0 Rallies Found</span>
backend\features\rally_seq.py:43:def _spread(a: np.ndarray) -> float:
backend\features\rally_seq.py:67:     play_is_x = _spread(x[mask]) >= _spread(y[mask])
backend\features\rally_seq.py:70:     deadband = 0.06 * (_spread(coord[mask]) if mask.sum() > 1
else 1.0)
backend\features\rally_seq.py:87:                if abs(c) > deadband and abs(pc) > deadband and
(c > 0) != (pc > 0):
docs\ANALYZERS_AND_RECONCILERS.md:81:finer-scope signals; everything ultimately lands as a per-
candidate feature column (or an event; the
docs\ANALYZERS_AND_RECONCILERS.md:116:`person_detector.py:256`), the net line
(`rally_gate.estimate_play_axis`, camera-agnostic), and the window
docs\ANALYZERS_AND_RECONCILERS.md:172:- **Logic:** the shuttle **enters the net region**
(`|coord ? net| < deadband`) **and** its velocity
docs\ANALYZERS_AND_RECONCILERS.md:402:- **Shadow-persist, then promote.** Signals/spans/issues
are persisted even while ignored, so a `Variant`
backend\pipeline\segmenters\fusion_hybrid.py:106:                self._axis_cache =
rally_gate.estimate_play_axis(points)
backend\pipeline\detectors\rally_gate.py:14:the visible points inside a candidate window, with a
deadband to ignore jitter.
backend\pipeline\detectors\rally_gate.py:37:def _spread(vals: Sequence[float]) -> float:
backend\pipeline\detectors\rally_gate.py:55:def estimate_play_axis(points) -> Tuple[str, float,
float]:
backend\pipeline\detectors\rally_gate.py:63:     sx, sy = _spread(xs), _spread(ys)
backend\pipeline\detectors\rally_gate.py:69:def count_net_crossings(window_points, axis: str,
net: float, deadband: float) -> int:
backend\pipeline\detectors\rally_gate.py:71:     deadband zone near the net (so jitter at the net
doesn't inflate the count)."""
backend\pipeline\detectors\rally_gate.py:78:                if abs(v) < deadband:
backend\pipeline\detectors\rally_gate.py:92:                min_crossings: int = 2,
deadband_frac: float = 0.06,
backend\pipeline\detectors\rally_gate.py:99:     `(axis, net, span)` (from `estimate_play_axis`)
so the  $O(P \log P)$  estimate isn't redone
backend\pipeline\detectors\rally_gate.py:101:     axis, net, span = estimate if estimate is not
None else estimate_play_axis(points)
backend\pipeline\detectors\rally_gate.py:102:     deadband = deadband_frac * span
backend\pipeline\detectors\rally_gate.py:106:     c = count_net_crossings(wp, axis, net,
deadband)
backend\pipeline\detectors\rally_gate.py:114:                min_crossings: int = 2,
deadband_frac: float = 0.06) -> List[Interval]:
backend\pipeline\detectors\rally_gate.py:117:                gate_windows(points, windows, fps,
min_crossings, deadband_frac) if g.kept]
tests\test_fusion_hybrid.py:26:     for f in range(300, 360):         # window B frames: y == net
level -> deadband -> no crossings
backend\eval\rally_seq_proto.py:46:     FEAT_NAMES, FEATURE_SCHEMA, FEATURE_SCHEMA_VERSION,
_spread,
backend\eval\fusion_golden.py:70:     axis, net_px, _span = rally_gate.estimate_play_axis(points)
backend\eval\serve_contrast.py:60:     est = rally_gate.estimate_play_axis(points)
tests\test_rally_gate.py:10:def test_estimate_play_axis_picks_larger_spread():
tests\test_rally_gate.py:12:     axis, net, span = rg.estimate_play_axis(pts)
tests\test_rally_gate.py:19:     assert rg.count_net_crossings(pts, "y", net=100.0, deadband=5.0)
== 3
tests\test_rally_gate.py:22:def test_deadband_ignores_jitter_near_net():
tests\test_rally_gate.py:23:     pts = [P(0, 0, 101), P(1, 0, 99), P(2, 0, 102), P(3, 0, 98)] #
all within deadband of net
tests\test_rally_gate.py:24:     assert rg.count_net_crossings(pts, "y", net=100.0,
deadband=10.0) == 0
tests\test_rally_gate.py:29:     assert rg.count_net_crossings(pts, "y", net=100.0, deadband=5.0)
== 0
tests\test_rally_gate.py:51:     est = rg.estimate_play_axis(pts)
tests\test_served_gate_a_regression.py:162:     est = rally_gate.estimate_play_axis(pts)
docs\CODE_MAP.md:161:[Omitted long matching line]

```

```
docs\mockups\ui_alpha_mockup.html:59: <div class="logo">? KhelSutra <span>Guru</span></div>
docs\mockups\ui_alpha_mockup.html:79: <tr><td><b>Tue evening doubles ? Court
2</b></td><td>today 18:42</td><td>?</td><td><span class="chip in">processing .
64%</span></td><td><button class="btn sm sec" onclick="go('job')">view</button></td></tr>
docs\mockups\ui_alpha_mockup.html:80: <tr><td><b>Singles vs Rohan (game
3)</b></td><td>yesterday</td><td>23</td><td><span class="chip ok">? ready</span></td><td><button
class="btn sm sec" onclick="go('review')">review</button></td></tr>
docs\mockups\ui_alpha_mockup.html:81: <tr><td><b>Academy U-19 trials cam A</b></td><td>Jun
8</td><td>31</td><td><span class="chip wn">?? ready, 2 notes</span></td><td><button class="btn
sm sec" onclick="go('report')">notes</button></td></tr>
docs\mockups\ui_alpha_mockup.html:82: <tr><td><b>phone_VID20260606.mp4</b></td><td>Jun
6</td><td>?</td><td><span class="chip er">? failed checks</span></td><td><button class="btn sm
sec" onclick="go('report')">why?</button></td></tr>
docs\mockups\ui_alpha_mockup.html:91: <span style="font-size:12px">uploads in chunks ? safe
to leave the tab open on slow connections</span></div>
docs\mockups\ui_alpha_mockup.html:108: <div class="stage"><span class="dot"
style="background:var(--acc)"></span> Upload received . 1.9 GB . checksum verified</div>
docs\mockups\ui_alpha_mockup.html:109: <div class="stage"><span class="dot"
style="background:var(--acc)"></span> Quality checks . resolution ? . fps ? . stability ? .
court visible ?</div>
docs\mockups\ui_alpha_mockup.html:110: <div class="stage"><span class="dot"
style="background:var(--acc2)"></span> <b>Finding rallies?</b> 64% ? 14 found so far</div>
docs\mockups\ui_alpha_mockup.html:111: <div class="stage"><span class="dot"
style="background:var(--line)"></span> Confirming with AI</div>
docs\mockups\ui_alpha_mockup.html:112: <div class="stage"><span class="dot"
style="background:var(--line)"></span> Quality report</div>
docs\mockups\ui_alpha_mockup.html:130: <span class="sub" style="margin:0">your corrections
improve your future results ? 18/23 confirmed</span>
docs\mockups\ui_alpha_mockup.html:132: <div class="rally"><span class="chip ok">? kept</span>
docs\mockups\ui_alpha_mockup.html:136: <div class="rally"><span class="chip in">new</span>
docs\mockups\ui_alpha_mockup.html:140: <div class="rally rej"><span class="chip er">?
rejected</span>
docs\mockups\ui_alpha_mockup.html:151: <span class="sub" style="margin:0">19 rallies . est.
6 m 40 s</span>
```

7. user (2026-06-19T16:40:22.361Z)

```
len2 [1,9] -> 0
len3 [1,5,9] -> 4
len10 0..9 -> 8
len20 0..19 -> 18
len21 0..20 -> 18
len2 lo idx 0 hi idx 0
len19 lo idx 0 hi idx 17
len20 lo idx 0 hi idx 18
len21 lo idx 1 hi idx 19
```

8. assistant (2026-06-19T16:40:26.115Z)

The repro confirms the index math. Now let me verify the downstream usage ? is `span`/`spread` ever divided by anything, and is `\_spread` ever called on small slices? Let me check the call sites in `fusion\_hybrid.py` and the deadband flow.

9. user (2026-06-19T16:40:27.198Z)

```
90         return stats
91
92         # P3 (learned fusion) first-step landing note (per MULTI_SIGNAL_FUSION_PLAN +
NEXT_STEPS 2026-06-14):
93         # Person features (from _person_features when cue_flags.person) are now persisted
for the harness
94         # `compare` LOVO litmus on the 6 golden videos. Eager-person-compute optimisation
(compute only
95         # for shuttle-seeded windows + padding, not whole video) is already in place via the
```

```

lazy
96         # self._person + windowed detections_per_frame. Next: actual harness run + lift
measurement
97         # (owner/hardware) before any served promotion. Flag-gated (default person OFF). See
98         # docs/NEXT_STEPS.md "Rally-detection quality track" and EXPERIMENT_HARNESS.md for
discipline.
99
100        def _axis_estimate(self, points: List[Any]) -> tuple:
101            """Cache rally_gate's (axis, net, span) play-axis estimate per trajectory so
it's
102            computed once per video, not once per candidate window (it depends only on
points)."""
103            key = id(points)
104            if self._axis_cache_key != key or self._axis_cache is None:
105                self._axis_cache_key = key
106                self._axis_cache = rally_gate.estimate_play_axis(points)
107            est = self._axis_cache
108            assert est is not None
109            return est
110
111        def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
112                             frame_width: float, video_path: str,
113                             fcfg: Dict[str, Any]) -> Dict[str, float]:
114            """GPU path (person cue ON): run the person detector over this window's
ORIGINAL-video
115            frame range and compute the scale/translation-invariant person features. Lazily
builds
116            ONE PersonDetector. ? real-video verification is owner-side; this wiring is
mock-tested."""
117            # Lazy import so the default (person-OFF) path never pulls torch/cv2 through
this module.
118            from backend.pipeline.detectors.person_detector import PersonDetector
119            if self._person is None:
120                self._person = PersonDetector(self.config)
121
122            frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
123            # round (not truncate): int() would map e.g. 1.999s*30 to frame 59 not 60 ? a
124            # window-edge off-by-one that drops the boundary frame from the shuttle
alignment.
125            start_f, end_f = round(cw["start"] * fps), round(cw["end"] * fps)
126            det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
127            fw = det["frame_width"] or frame_width
128            fh = det["frame_height"]
129            # If we can't trust the frame dims, every normalised feature would silently
collapse

```

10. user (2026-06-19T16:40:27.715Z)

```

40        from backend.eval.calibrate_wasb import load_golden_gts
41        from backend.eval.metrics import score
42        # Feature extraction is single-sourced in backend/features (ADR-008): training and
43        # serving import the SAME code, so train == serve by construction. Names re-exported
44        # here for back-compat with existing imports/tests.
45        from backend.features.rally_seq import ( # noqa: F401 (re-exports)
46            FEAT_NAMES, FEATURE_SCHEMA, FEATURE_SCHEMA_VERSION, _spread,
47            build_frame_arrays, features,
48        )
49
50        Interval = Tuple[float, float]
51
52
53        def labels_for(times: np.ndarray, gts: List[Interval]) -> np.ndarray:
54            y = np.zeros(len(times))

```

11. assistant (2026-06-19T16:40:34.599Z)

I have a complete picture. The `rally\_seq\_proto.py:46` import re-exports the `\_spread` from `rally\_seq.py` (numpy version), NOT the `rally\_gate.py` one. Let me confirm there is no division by `span` anywhere by checking how `deadband` is used ? it's a `\* span` multiplication and `count\_net\_crossings` only uses `abs(v) < deadband`. Let me also confirm `estimate\_play\_axis` is only ever called on full-trajectory `points`, never on window slices.

The evidence is complete. Let me verify the one remaining question: does `count\_net\_crossings` divide by deadband anywhere, or just compare? Already confirmed from the read: line 78 `if abs(v) < deadband: continue` ? pure comparison, no division. And `gate\_windows` line 102 `deadband = deadband\_frac \* span` ? multiplication. No division by spread anywhere.

Let me check the test file to confirm there's no production small-slice usage I'm missing.

12. user (2026-06-19T16:40:35.628Z)

```
40         assert not gated[1].kept and gated[1].crossings <= 1
41         assert rg.apply_gate(pts, windows, fps, min_crossings=2) == [(0.0, 1.0)]
42
43
44     def test_gate_windows_precomputed_estimate_matches_auto():
45         """Passing a precomputed (axis,net,span) estimate (the per-video reuse the fusion
46         segmenter does) yields identical results to letting gate_windows re-estimate."""
47         fps = 10.0
48         pts = [P(i, 500, 50 if i % 2 == 0 else 250) for i in range(0, 10)] + [P(20, 500,
49         50)]
50         windows = [(0.0, 1.0), (1.5, 2.5)]
51         auto = rg.gate_windows(pts, windows, fps, min_crossings=2)
52         est = rg.estimate_play_axis(pts)
53         reused = rg.gate_windows(pts, windows, fps, min_crossings=2, estimate=est)
54         assert [(g.crossings, g.kept) for g in auto] == [(g.crossings, g.kept) for g in
55         reused]
```

13. assistant (2026-06-19T16:40:37.622Z)

I have everything I need. Let me record the verdict.

14. user (2026-06-19T16:41:13.416Z)

Structured output provided successfully

15. assistant (2026-06-19T16:41:23.992Z)

I've completed the adversarial review and returned the structured verdict.

Summary: The claim is **misleading** and **not a real bug** (cosmetic severity).

- The quoted code and the arithmetic facts are accurate: `int()` truncates, and for `len(s) == 2` the spread is exactly `0.0` (verified by repro: `\_spread([1,9]) -> 0`). The p95 only reaches the true last index at `len >= 20`.
- But the impact framing is wrong. `\_spread` (`backend/pipeline/detectors/rally\_gate.py:37`) is only ever reached via `estimate\_play\_axis`, which is always called on the **whole video's** visible trajectory (`fusion\_hybrid.py:106`, `eval/fusion\_golden.py:70`, `eval/serve\_contrast.py:60`) ? never on small window slices. Windowing happens downstream after the scalar `span` is already computed.
- The "divide-by-zero / zero-deadband crash" is fabricated: `span` feeds a **multiplication** (`deadband = deadband\_frac \* span`, line 102) and a **comparison** (`abs(v) < deadband`, line 78). No division by spread exists anywhere. A zero deadband is harmless ? it just disables jitter suppression.
- There's a **second, independent** `\_spread` at `backend/features/rally\_seq.py:43` using non-truncating `np.percentile`, and that's the one actually feeding the Gen-0 TAL feature schema (re-exported at `rally\_seq\_proto.py:46`).
- The proposed `int()->math.ceil()` fix is risky, not safe: it would shift the p5/p95 indices on the real production estimate, changing `span` ? `deadband` ? crossing counts ? kept/dropped

rally decisions on every video ? a Launch-Report-gated behavior change with no measured benefit.